

MILPÍN

Sistema de decisión de Soporte Agrícola Inteligente

Valle del Yaqui — Modulo 3, DR-041 — Sonora, Mexico

ESPECIFICACION DE REQUERIMIENTOS FUNCIONALES Y NO FUNCIONALES

Version: 1.0 | Fecha: 20 de mayo de 2026

Estado del Proyecto: MVP — Prototipo Funcional

1. introducción

El presente documento especifica los requerimientos funcionales y no funcionales del sistema MILPIN, un DSS (Decision Support System) Agrícola inteligente diseñado para optimizar el uso del agua de riego en el Valle del Yaqui, Sonora, Mexico (Distrito de Riego 041, Modulo 3).

Los requerimientos funcionales se presentan en formato de Casos de Uso Expandidos siguiendo la metodología del Capítulo 6 del libro de texto de referencia. Cada caso de uso detalla el actor primario, el flujo de eventos típico y alternativo, reglas de negocio y aspectos abiertos.

Los requerimientos no funcionales se presentan en formato tabular con ID, categoría, descripción y métricas de aceptación verificables.

1.1 KPI Central del Proyecto

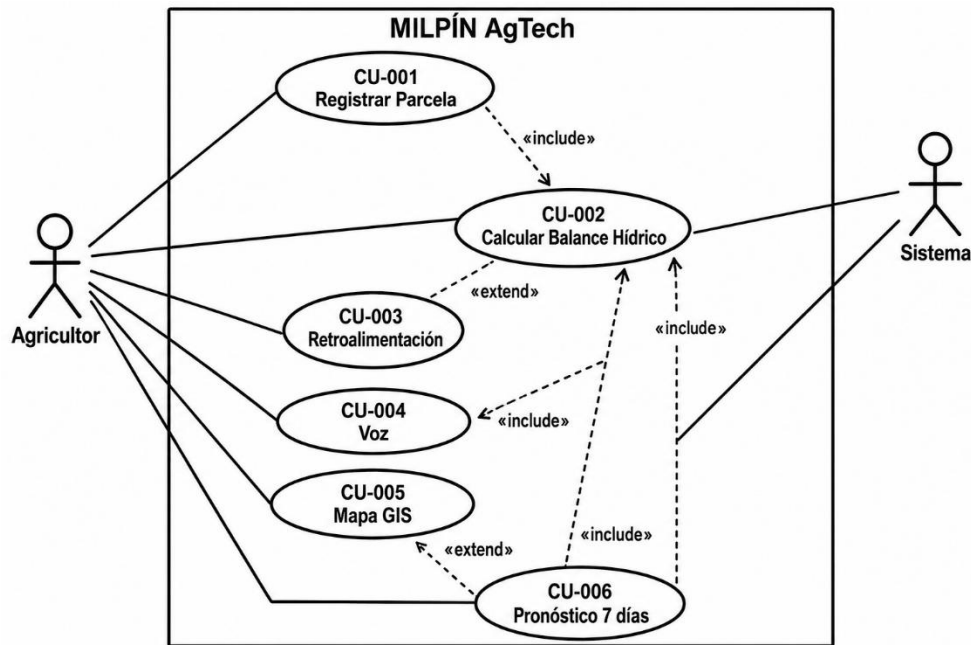
Reducir el consumo de agua de riego de 8,000 m3/ha/ciclo a 6,000 m3/ha/ciclo (ahorro objetivo del 25%) usando el motor FAO-56 Penman-Monteith, análisis de datos y recomendaciones automatizadas. Tarifa base: \$1.68 MXN/m3 (CFE 9-CU, bombeo 80 m).

1.2 Estado Actual del Sistema (2026-05-20)

Pre-MVP con core técnico sólido. PostgreSQL 15 + PostGIS 3.6 operativos con 7 modelos ORM y 14 endpoints CRUD. Motor FAO-56 conectado a BD (2026-04-25). Propagación de balance hídrico real sustituye humedad inicial ficticia (2026-05-06). Pipeline de voz (Whisper -> Ollama -> TTS) funcional. 77+ tests activos.

2. Requerimientos Funcionales — Casos de Uso Expandidos

A continuación se presentan los seis casos de uso principales del sistema MILPIN siguiendo el formato de narración expandida (Figura 6.13, Capítulo 6 del libro de referencia). Los números (1-10) corresponden a las secciones estándar del formato de caso de uso.



2.1 CU-MILP-001 — Registrar y Gestionar Parcela

Nombre del caso de uso:	Registrar y Gestionar Parcela	Requerimiento de Negocios: B
ID del caso de uso:	CU-MILP-001	
Prioridad:	Alta	
Fuente:	Entrevistas con agricultores del Modulo 3, DR-041	
Actor primario de negocios:	Agricultor (usuario del sistema)	
Otros actores participantes:	Administrador del Modulo (receptor externo) Sistema PostGIS (receptor externo)	
Otros involucrados interesados:	Coordinación del Modulo 3: interesados en el inventario de parcelas y su georreferenciación. Administración DR-041: interesados en la cobertura territorial del sistema.	
descripción:	Este caso de uso describe el registro de una nueva parcela agrícola en MILPIN. El agricultor proporciona datos geoespaciales (polígono GIS), cultivo asignado, área en hectáreas y parámetros de suelo. El sistema valida y persiste la geometría en PostGIS (GEOMETRY Polygon, SRID 4326) mediante el índice GIST activo. Al completarse, la parcela queda disponible en el mapa Leaflet y en los cálculos agronómicos FAO-56.	
1 Precondicion:	El agricultor debe estar registrado en el sistema con un id_usuario valido (UUID).	
2 Ocasionador:	El caso de uso se inicia cuando el agricultor selecciona "Nueva Parcela" en la interfaz o dicta el comando de voz equivalente.	
3 Curso típico de eventos:	Accion del actor	Respuesta del sistema

	Paso 1: El agricultor ingresa los datos de la parcela: nombre, superficie (ha), cultivo, capacidad de campo (CC), punto de marchitez permanente (PMP) y delimita el poligono en el mapa.	Paso 2: El sistema valida que los datos requeridos estén completos y que el polígono es geoméricamente valido (shapely make_valid).
		Paso 3: El sistema proyecta el polígono al SRID 4326 y calcula el centroide para el mapa.
		Paso 4: El sistema persiste la parcela en la tabla "parcelas" con geom GEOMETRY(Polygon,4326) y registra la fecha de creación.
		Paso 5: El sistema expone la parcela vía GET /api/parcelas/geojson para visualización en Leaflet.
		Paso 6: El sistema confirma el registro y actualiza el mapa con el nuevo poligono.
4 Cursos alternos:	Paso alternativo 2a: Si el polígono es invalido geoméricamente, el sistema aplica Douglas-Peucker y make_valid. Paso alternativo 2b: Si faltan datos obligatorios (CC, PMP, cultivo), el sistema senala los campos incompletos.	
5 Conclusion:	El caso de uso concluye cuando la parcela queda registrada en BD y visible en el mapa GIS.	
6 Postcondicion:	La parcela esta en la tabla "parcelas" con geometria PostGIS valida. El endpoint /api/parcelas/geojson incluye la nueva Feature.	
7 Reglas de negocios:	Los valores de CC y PMP deben cumplir $0 < PMP < CC \leq 1.0$ (m3/m3). El cultivo asignado debe pertenecer al catalogo oficial: Maiz, Frijol, Algodon, Uva o Chile.	
8 Restricciones y especificaciones de implantacion:	La captura del poligono se opera exclusivamente desde el mapa Leaflet (sin frameworks adicionales). El sistema debe usar el endpoint POST /api/parcelas del router FastAPI existente.	
9 Hipotesis:	Se asume que el agricultor tiene acceso a internet para cargar las capas Esri World Imagery durante el registro.	
10 Aspectos abiertos:	1. Definir si el poligono puede importarse desde SHP o KML, ademas del dibujo manual. 2. Establecer limite maximo de hectareas por parcela registrable en el sistema.	

2.2 CU-MILP-002 — Calcular Balance Hidrico y Generar Recomendacion de Riego

Nombre del caso de uso:	Calcular Balance Hidrico y Generar Recomendacion de Riego	Requerimiento de Negocios: B
ID del caso de uso:	CU-MILP-002	
Prioridad:	Alta	
Fuente:	Motor agronomico FAO-56, Allen et al. 1998; entrevistas con tecnicos del Modulo 3	
Actor primario de negocios:	Sistema MILPIN (procesamiento automatico)	
Otros actores participantes:	Agricultor (receptor externo) Servicio NASA POWER (servicio externo de datos climaticos)	
Otros involucrados interesados:	Administracion DR-041: interesados en reduccion de consumo hidrico (KPI: 8,000 -> 6,000 m3/ha/ciclo). Tecnicos agronomicos: interesados en la precision del modelo FAO-56.	
Descripcion:	Este caso de uso describe el calculo automatico del balance hidrico diario usando el metodo FAO-56 Penman-Monteith (con Hargreaves como fallback). El sistema lee datos climaticos de "clima_diario", obtiene parametros del cultivo (Kc por etapa fenologica) y del suelo desde la parcela, propaga el balance acumulado desde el ultimo riego real (propagar_balance_hidrico()), calcula el deficit hidrico y genera y persiste una recomendacion de riego en la tabla "recomendaciones".	
1 Precondicion:	La parcela debe tener cultivo asignado, parametros de suelo (CC, PMP) registrados y al menos un registro en "historial_riego" para inicializar la humedad via propagar_balance_hidrico().	
2 Ocasionador:	El caso de uso se inicia cuando el agricultor consulta el balance hidrico (GET /api/balance_hidrico/{parcela_id}) o cuando el sistema ejecuta el recalcu lo diario programado.	
3 Curso tipico de eventos:	Accion del actor	Respuesta del sistema
	Paso 1: El agricultor selecciona una parcela o consulta por voz el estado hidrico.	Paso 2: El sistema recupera los datos de la parcela, cultivo y parametros de suelo desde la BD.
		Paso 3: El sistema recupera registros de "clima_diario" (tmax, tmin, HR, radiacion, viento).
		Paso 4: El sistema calcula ETo diaria con Penman-Monteith FAO-56. Si faltan variables, activa Hargreaves.
		Paso 5: El sistema aplica Kc (KC_TABLE en balance_hidrico.py) segun etapa fenologica calculada a partir de dias_siembra.
		Paso 6: El sistema propaga el balance dia a dia desde el ultimo riego real con propagar_balance_hidrico(), eliminando la humedad inicial ficticia (CC+PMP)/2.

		Paso 7: El sistema calcula el deficit hidrico y determina si supera el umbral de riego (50% del agua disponible total).
		Paso 8: El sistema persiste la recomendacion en "recomendaciones" con lamina_recomendada, fecha y estado="pendiente".
		Paso 9: El sistema muestra al agricultor: deficit actual, lamina recomendada (mm), costo estimado (\$MXN) y proyeccion 7 dias.
4 Cursos alternos:	Paso alternativo 4a: Si clima_diario no tiene registros recientes, el sistema activa el ingestor NASA POWER ETL. Paso alternativo 6a: Sin historial_riego, se usa $(CC + PMP)/2$ y se notifica baja confianza de la estimacion. Paso alternativo 7a: Si el deficit no supera el umbral, se genera recomendacion "no regar" con fecha estimada de proximo riego.	
5 Conclusion:	El caso de uso concluye cuando la recomendacion esta generada y persistida en BD con el calculo FAO-56 completo.	
6 Postcondicion:	La tabla "recomendaciones" contiene el nuevo registro. El agricultor puede visualizar el deficit y la lamina en la UI.	
7 Reglas de negocios:	La ETo se calcula con FAO-56 Penman-Monteith; Hargreaves solo se activa si faltan ≥ 2 variables climaticas. La tarifa base es \$1.68 MXN/m³ (CFE 9-CU, bombeo 80 m) para el calculo de costo. KPI del sistema: reducir consumo de 8,000 m³/ha/ciclo a 6,000 m³/ha/ciclo (ahorro 25%).	
8 Restricciones y especificaciones de implantacion:	El motor FAO-56 debe ejecutarse en backend/core/balance_hidrico.py sin dependencias de librerias agronomicas externas. El calculo debe completarse en menos de 3 segundos por parcela.	
9 Hipotesis:	Se asume que los datos de NASA POWER tienen latencia maxima de 24 horas y que los parametros Kc de KC_TABLE son validos para el Valle del Yaqui.	
10 Aspectos abiertos:	1. Validar los coeficientes Kc con datos reales de campo del Modulo 3. 2. Determinar la frecuencia optima de recalcu automatico (diario vs. cada 6 horas).	

2.3 CU-MILP-003 — Registrar Retroalimentacion del Agricultor sobre Recomendacion

Nombre del caso de uso:	Registrar Retroalimentacion del Agricultor sobre Recomendacion	Requerimiento de Negocios: B
ID del caso de uso:	CU-MILP-003	
Prioridad:	Alta	
Fuente:	Ciclo de mejora continua; feedback de campo con agricultores del Modulo 3	
Actor primario de negocios:	Agricultor (usuario del sistema)	
Otros actores participantes:	Tabla historial_riego (receptor interno de BD) Modelo ETo Ridge Regression (receptor interno)	
Otros involucrados interesados:	Administracion DR-041: interesados en el cumplimiento real de riegos recomendados. Equipo de ML: interesados en datos de feedback para reentrenamiento del modelo.	
Descripcion:	Este caso de uso describe el proceso mediante el cual el agricultor retroalimenta una recomendacion de riego previamente generada. El agricultor puede aceptar, rechazar o modificar la lamina recomendada. Si acepta o modifica, el sistema persiste automaticamente un registro en "historial_riego" con la lamina real aplicada, cerrando el loop de aprendizaje. Resuelto 2026-05-01 (TestFeedbackLoop, 7 casos verificados).	
1 Precondicion:	Debe existir una recomendacion de riego con estado="pendiente" para la parcela del agricultor.	
2 Ocasionador:	El caso de uso se inicia cuando el agricultor selecciona una recomendacion pendiente y decide si aplicar o no el riego.	
3 Curso tipico de eventos:	Accion del actor	Respuesta del sistema
	Paso 1: El agricultor consulta la lista de recomendaciones pendientes para su parcela.	Paso 2: El sistema recupera las recomendaciones con estado="pendiente" y las presenta al agricultor.
	Paso 3: El agricultor elige "Aceptar", "Rechazar" o "Modificar" la lamina recomendada.	Paso 4: El sistema valida la accion. Si es "Modificar", valida que la lamina sea un valor positivo en mm.
		Paso 5: Si la accion es "aceptada" o "modificada", el sistema auto-inserta un registro en "historial_riego" con lamina real, fecha y parcela_id.
		Paso 6: El sistema actualiza el estado de la recomendacion a "aceptada", "rechazada" o "modificada".
		Paso 7: El sistema recalcula la humedad inicial con propagar_balance_hidrico() usando el nuevo dato de riego real.
		Paso 8: El sistema confirma al agricultor el registro y actualiza el

		dashboard con el nuevo estado de la parcela.
4 Cursos alternos:	Paso alternativo 4a: Si el agricultor ingresa lamina de 0 mm o negativa al modificar, el sistema solicita un valor positivo. Paso alternativo 5a: Si falla la insercion en historial_riego, el sistema revierte el cambio de estado y notifica el error.	
5 Conclusion:	El caso de uso concluye cuando la retroalimentacion queda registrada y historial_riego actualizado, cerrando el loop de aprendizaje.	
6 Postcondicion:	La tabla "recomendaciones" refleja el estado actualizado. "historial_riego" contiene el registro de riego real cuando aplica. El proximo calculo FAO-56 usa la humedad actualizada.	
7 Reglas de negocios:	Solo se registra en historial_riego cuando la retroalimentacion es "aceptada" o "modificada". La lamina modificada debe ser positiva y no superior a 3x la lamina recomendada por el sistema (limite de coherencia agronomica).	
8 Restricciones y especificaciones de implantacion:	El endpoint PATCH /api/recomendaciones/{id}/feedback debe procesar en una transaccion atomica (recomendacion + historial_riego en el mismo commit de BD).	
9 Hipotesis:	Se asume que el agricultor registra el feedback el mismo dia en que aplica el riego, con margen de +-24 horas.	
10 Aspectos abiertos:	1. Definir si el rechazo sistematico de recomendaciones debe generar una alerta al tecnico agronomico del modulo. 2. Evaluar si el historial de feedback debe alimentar el reentrenamiento automatico del modelo XGBoost (Prefect Fase C/D).	

2.4 CU-MILP-004 — Procesar Comando de Voz en Espanol

Nombre del caso de uso:	Procesar Comando de Voz en Espanol	Requerimiento de Negocios: B
ID del caso de uso:	CU-MILP-004	
Prioridad:	Media	
Fuente:	Requerimiento de usabilidad; agricultores con baja alfabetizacion digital en el Valle del Yaqui	
Actor primario de negocios:	Agricultor (usuario del sistema)	
Otros actores participantes:	Whisper STT (servicio externo, receptor de audio) Ollama llama3.2:latest (servicio local de NLU) Web Speech API (síntesis de voz TTS)	
Otros involucrados interesados:	Administracion DR-041: interesados en adopcion del sistema por agricultores con bajo dominio tecnologico.	
Descripcion:	Este caso de uso describe el flujo completo de procesamiento de comandos de voz en espanol. El agricultor graba un audio corto; el sistema lo transcribe con Whisper (modelo "base"), clasifica la intencion con Ollama (llama3.2:latest), extrae entidades (parcela, cultivo, fecha) y ejecuta la accion correspondiente. La respuesta se sintetiza en voz usando Web Speech API. Pipeline: Audio -> Whisper STT -> Ollama NLU -> JSON intent -> accion MILPIN -> TTS.	
1 Precondicion:	El servicio Ollama debe estar activo con el modelo llama3.2:latest cargado. El usuario debe conceder permisos de microfono al navegador.	
2 Ocasionador:	El caso de uso se inicia cuando el agricultor presiona el boton de microfono y graba un comando en espanol.	
3 Curso típico de eventos:	Accion del actor	Respuesta del sistema
	Paso 1: El agricultor presiona el boton de microfono y dicta un comando (ej. "Cuanta agua necesita mi parcela de maiz?").	Paso 2: El sistema recibe el audio y lo envia al endpoint POST /api/voz/transcribir. Whisper (modelo base) transcribe el audio a texto.
		Paso 3: El sistema envia la transcripcion a Ollama (llama3.2:latest) para clasificacion de intencion y extraccion de entidades. Devuelve JSON intent.
		Paso 4: El sistema valida el intent contra VALID_CULTIVOS (Maiz, Frijol, Algodon, Uva, Chile) y las parcelas registradas del agricultor.
		Paso 5: El sistema ejecuta la accion correspondiente al intent y obtiene la respuesta del modulo correspondiente.
		Paso 6: El sistema sintetiza la respuesta en voz con Web Speech API (TTS) y la reproduce al agricultor.

	Paso 7: El agricultor confirma si la accion ejecutada es la correcta.	Paso 8: El sistema registra el evento de voz y actualiza la interfaz con el resultado.
4 Cursos alternos:	Paso alternativo 2a: Si Whisper no puede transcribir el audio (ruido excesivo), solicita al agricultor repetir el comando. Paso alternativo 3a: Si Ollama no puede clasificar la intencion (confianza < umbral), pide aclaracion por voz. Paso alternativo 4a: Si el cultivo o parcela mencionada no existe en el sistema, informa las opciones disponibles.	
5 Conclusion:	El caso de uso concluye cuando la accion solicitada por voz fue ejecutada y confirmada al agricultor mediante sintesis de voz.	
6 Postcondicion:	La accion del intent fue ejecutada en el sistema. El evento queda registrado en el log de comandos de voz.	
7 Reglas de negocios:	Los comandos de voz solo ejecutan acciones de consulta y visualizacion; las modificaciones de datos requieren confirmacion textual adicional. El sistema solo procesa comandos en espanol.	
8 Restricciones y especificaciones de implantacion:	ADVERTENCIA: voice_endpoint.py tiene una vulnerabilidad activa de path traversal: temp_path = f"temp_{audio_file.filename}" sin sanitizacion. PENDIENTE DE CORRECCION (Deuda Tecnica Fase A, item 2). El archivo de audio no debe superar 10 MB y debe ser de tipo audio/wav, audio/mp3 o audio/webm.	
9 Hipotesis:	Se asume que Ollama esta ejecutandose localmente en el mismo servidor que FastAPI, con latencia < 2 segundos por request.	
10 Aspectos abiertos:	1. Sanitizar path traversal en voice_endpoint.py (alta prioridad de seguridad). 2. Definir el catalogo completo de intents validos y su cobertura de casos de uso.	

2.5 CU-MILP-005 — Visualizar Mapa GIS de Parcelas

Nombre del caso de uso:	Visualizar Mapa GIS de Parcelas	Requerimiento de Negocios: B
ID del caso de uso:	CU-MILP-005	
Prioridad:	Alta	
Fuente:	Requerimiento de visualizacion geoespacial; Modulo 3, DR-041	
Actor primario de negocios:	Agricultor / Administrador del Modulo	
Otros actores participantes:	API PostGIS / GeoJSON estatico (receptor externo) Leaflet 1.9.4 (cliente de mapas) Esri World Imagery + OpenTopoMap (capas base externas)	
Otros involucrados interesados:	Coordinacion Modulo 3: interesados en distribucion espacial y estado de riego por parcela. Administracion DR-041: interesados en la cobertura geografica del sistema.	
Descripcion:	Este caso de uso describe la visualizacion interactiva de las parcelas del Modulo 3 en el mapa Leaflet. El sistema carga los poligonos desde GET /api/parcelas/geojson (PostGIS) o desde el fallback estatico lotes.geojson. Cada parcela se muestra con estado de riego (color-coded), informacion de cultivo y ultimo balance hidrico. El usuario puede hacer clic en una parcela para ver el detalle y acceder a las recomendaciones.	
1 Precondicion:	El usuario debe tener acceso al frontend de MILPIN. El backend FastAPI debe estar activo.	
2 Ocasionador:	El caso de uso se inicia cuando el usuario abre la aplicacion MILPIN en el navegador y el mapa Leaflet carga.	
3 Curso tipico de eventos:	Accion del actor	Respuesta del sistema
	Paso 1: El usuario abre MILPIN. El mapa Leaflet inicializa con las capas base Esri World Imagery y OpenTopoMap.	Paso 2: map_engine.js ejecuta GET /api/parcelas/geojson para obtener la FeatureCollection con los poligonos de parcelas.
		Paso 3: El sistema construye la FeatureCollection desde la tabla "parcelas" usando ST_AsGeoJSON de PostGIS sobre la columna geom.
		Paso 4: El sistema agrega a cada Feature las propiedades: parcela_id, nombre, cultivo, superficie_ha, ultimo_balance, estado_riego.
		Paso 5: Leaflet renderiza los poligonos con color-coding: verde (optimo), amarillo (deficit moderado), rojo (deficit critico).
	Paso 6: El usuario hace clic en una parcela del mapa.	Paso 7: El sistema muestra un popup con: nombre, cultivo, superficie, deficit hidrico actual y ultima recomendacion.
	Paso 8: El usuario navega a la vista detallada desde el popup.	Paso 9: El sistema carga la vista de detalle con el balance hidrico,

	historial de riegos y recomendaciones activas.
4 Cursos alternos:	Paso alternativo 2a: Si /api/parcelas/geojson no responde, map_engine.js activa el fallback y carga lotes.geojson estatico desde frontend/data/. Paso alternativo 5a: Si un poligono tiene geometria invalida en BD, se omite de la FeatureCollection y se registra en el log.
5 Conclusion:	El caso de uso concluye cuando todos los poligonos de parcelas son visibles en el mapa Leaflet con su estado de riego actualizado.
6 Postcondicion:	El mapa muestra las parcelas con informacion de estado de riego en tiempo real. El agricultor puede interactuar con cualquier parcela.
7 Reglas de negocios:	El color de la parcela debe reflejar el estado del balance hidrico mas reciente (≤ 24 horas). Las capas de mapa base deben incluir siempre Esri World Imagery como opcion de alta resolucion.
8 Restricciones y especificaciones de implantacion:	El mapa debe implementarse exclusivamente con Leaflet 1.9.4 (sin React-Leaflet ni otros wrappers). La carga inicial del GeoJSON debe completarse en menos de 5 segundos con hasta 500 parcelas.
9 Hipotesis:	Se asume que los usuarios acceden con una conexion minima de 1 Mbps para cargar las capas de imagen satelital.
10 Aspectos abiertos:	1. Definir si se implementara clustering visual de parcelas (Leaflet.markercluster) en zoom bajo. 2. Evaluar integracion de capas adicionales: canales de riego, zonas de acuífero, limites del Modulo 3.

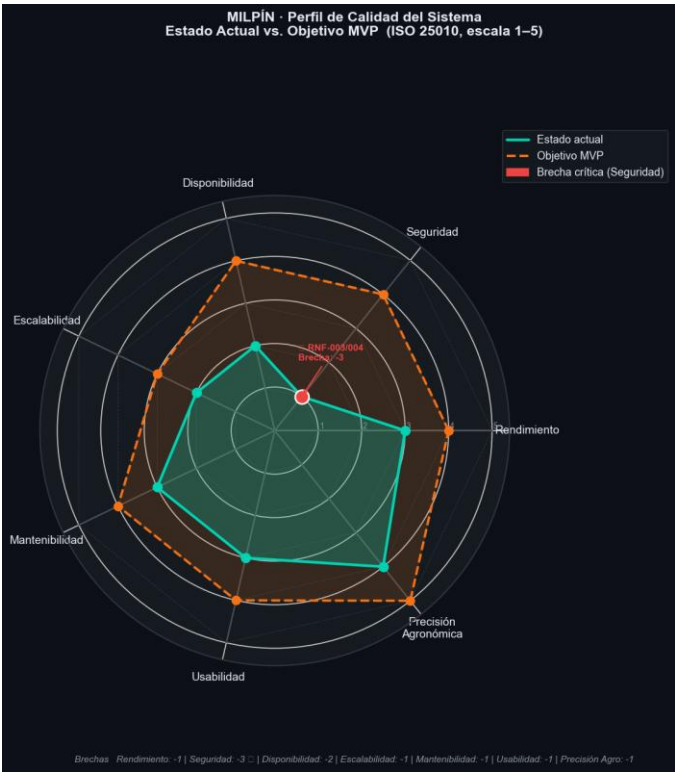
2.6 CU-MILP-006 — Consultar Pronostico ETo y Proyeccion de Proximo Riego (7 dias)

Nombre del caso de uso:	Consultar Pronostico ETo y Proyeccion de Proximo Riego (7 dias)	Requerimiento de Negocios: B
ID del caso de uso:	CU-MILP-006	
Prioridad:	Media	
Fuente:	Modulo ML/forecasting MILPIN; requerimiento de planificacion agronomica	
Actor primario de negocios:	Agricultor (usuario del sistema)	
Otros actores participantes:	Modulo eto_forecast.py / Ridge Regression (receptor interno) Tabla clima_diario (receptor interno de BD)	
Otros involucrados interesados:	Tecnicos agronomicos: interesados en precision del pronostico para planificacion de insumos. Administracion DR-041: interesados en anticipacion a eventos de deficit hidrico.	
Descripcion:	Este caso de uso describe la consulta del pronostico de ETo para los proximos 7 dias y la proyeccion de la fecha estimada del proximo riego. El sistema usa un modelo Ridge Regression (eto_forecast.py) sobre "clima_diario" con features sin/cos del dia del ano y lags de ETo y temperatura maxima. La proyeccion corre FAO-56 forward sobre el pronostico de ETo para estimar cuando el deficit alcanzara el umbral. Endpoint: GET /api/parcelas/{id}/forecast?dias_siembra=N&horizon=7.	
1 Precondicion:	La parcela debe tener cultivo asignado y al menos 60 registros en "clima_diario" para que Ridge Regression opere en modo entrenado (fallback: media movil 14 dias).	
2 Ocasionador:	El caso de uso se inicia cuando el agricultor accede al tab "Pronostico 7 dias" en la interfaz o solicita por voz la proyeccion de riego.	
3 Curso tipico de eventos:	Accion del actor	Respuesta del sistema
	Paso 1: El agricultor navega al tab "Riego" y selecciona "Proyeccion 7 dias" para su parcela.	Paso 2: El sistema ejecuta GET /api/parcelas/{id}/forecast?dias_siembra=N&horizon=7 en el backend.
		Paso 3: eto_forecast.py extrae features (sin/cos doy, lags ETo, t_max) desde clima_diario y genera la proyeccion de ETo para 7 dias.
		Paso 4: Si la BD tiene < 60 registros, el sistema activa el fallback de media movil 14 dias y notifica la baja confianza del pronostico.
		Paso 5: El sistema corre FAO-56 forward con el ETo proyectado, aplicando Kc por etapa fenologica, para simular el balance hidrico futuro.
		Paso 6: El sistema calcula la fecha estimada en que el deficit alcanzara el umbral de riego (+-dias de incertidumbre).
		Paso 7: El sistema devuelve el timeline de deficit diario (7 puntos) y la fecha estimada de proximo riego al frontend.
	Paso 8: El agricultor visualiza el timeline en la seccion "Proyeccion 7 dias" del tab Riego.	Paso 9: El sistema resalta la fecha estimada de proximo riego y el deficit acumulado proyectado en mm.

4 Cursos alternos:	Paso alternativo 3a: Sin datos climaticos recientes (72 horas), el sistema notifica que el pronostico puede estar desactualizado y activa el ingestor NASA POWER ETL. Paso alternativo 6a: Si el balance proyectado no muestra deficit en 7 dias, el sistema informa "No se requiere riego en el horizonte de 7 dias".
5 Conclusion:	El caso de uso concluye cuando el agricultor visualiza el timeline de ETo proyectada y la fecha estimada del proximo riego.
6 Postcondicion:	El agricultor cuenta con informacion de planificacion hidrica para los proximos 7 dias. El resultado del pronostico queda cacheado para el mismo dia.
7 Reglas de negocios:	Ridge Regression (eto_forecast.py) solo opera en modo entrenado si hay ≥ 60 registros en clima_diario; en caso contrario usa fallback de media(14d). La proyeccion debe mostrar explicitamente el nivel de incertidumbre (fallback vs. modelo entrenado).
8 Restricciones y especificaciones de implantacion:	eto_forecast.py debe residir en backend/core/eto_forecast.py y no importar codigo del directorio ml/training/. El calculo del pronostico completo (Ridge + FAO-56 forward) debe ejecutarse en menos de 5 segundos.
9 Hipotesis:	Se asume que Ridge Regression es suficientemente preciso para horizontes de 7 dias en el Valle del Yaqui. La validacion con datos reales de campo esta pendiente.
10 Aspectos abiertos:	1. Migrar el entrenamiento del modelo Ridge a ml/training/eto_ridge/train.py (stub actualmente, Fase B). 2. Evaluar si se requiere un modelo mas sofisticado (LSTM, Prophet) para el horizonte de 7 dias. 3. Validar el pronostico contra datos historicos reales del DR-041.

3. Requerimientos No Funcionales

Los requerimientos no funcionales definen los atributos de calidad del sistema MILPIN. Se incluyen requerimientos de rendimiento, seguridad, disponibilidad, escalabilidad, mantenibilidad, usabilidad y precision agronomica. Cada requerimiento incluye una metrica de aceptacion verificable.

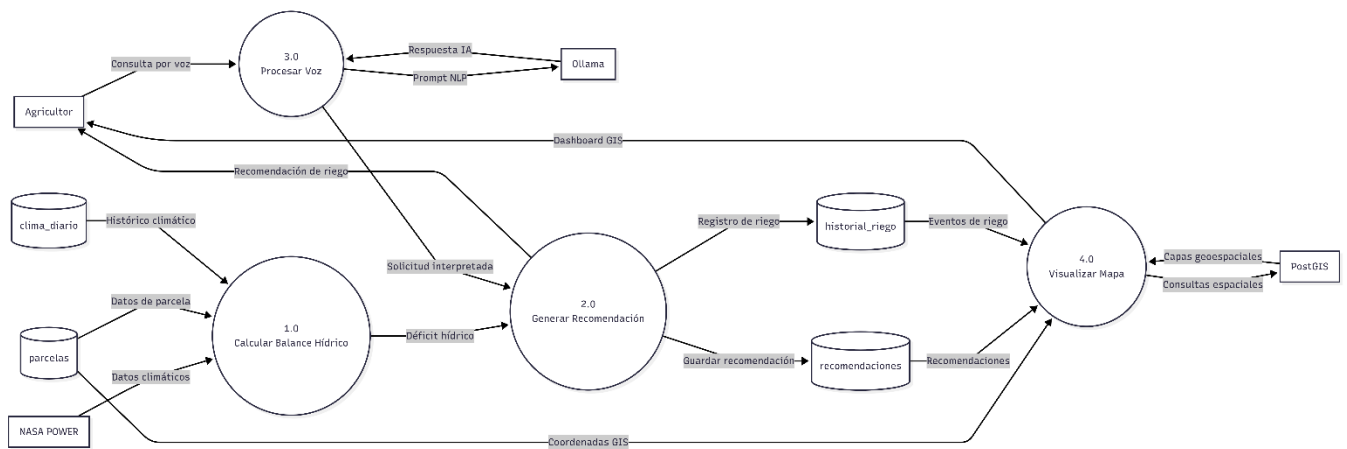


ID	Categoria	Descripcion del Requerimiento	Metrica / Umbral de Aceptacion	Prioridad
RNF-001	Rendimiento	El calculo del balance hidrico FAO-56 para una parcela debe completarse en menos de 3 segundos bajo condiciones normales de operacion.	Tiempo respuesta <= 3 s (p95) para GET /api/balance_hidrico/{id}	Alta
RNF-002	Rendimiento	La carga inicial del mapa GIS con hasta 500 parcelas (FeatureCollection GeoJSON) debe completarse en menos de 5 segundos.	Tiempo carga mapa <= 5 s con 500 Features en conexion de 1 Mbps	Alta
RNF-003	Seguridad	Las credenciales de PostgreSQL no deben estar en texto plano en archivos versionados. El archivo backend/.env debe estar en .gitignore.	0 credenciales expuestas en commits del repositorio. Validar con: git log --all -S "password"	Critica
RNF-004	Seguridad	El endpoint voice_endpoint.py debe sanitizar el nombre del archivo de audio antes de crear archivos temporales, previniendo ataques de path traversal.	Nombre de archivo temporal generado con uuid.uuid4(). 0 vulnerabilidades de path traversal detectadas.	Critica
RNF-005	Seguridad	El backend debe restringir los origenes CORS a una lista blanca de dominios autorizados. No se permite allow_origins=["*"] en produccion.	La configuracion CORS en main.py contiene allowlist explicita en entornos no-localhost.	Alta

RNF-006	Disponibilidad	El sistema backend (FastAPI + PostgreSQL) debe estar disponible al menos el 95% del tiempo durante la temporada de cultivo activa (octubre-mayo).	<i>Uptime >= 95% en periodo octubre-mayo. RTO <= 30 minutos ante falla.</i>	Alta
RNF-007	Disponibilidad	El mapa Leaflet debe funcionar en modo degradado (GeoJSON estatico desde frontend/data/lotos.geojson) cuando el backend no este disponible.	<i>La interfaz de mapa carga y muestra parcelas estaticas sin conexion al backend.</i>	Media
RNF-008	Escalabilidad	El sistema debe soportar al menos 50 usuarios concurrentes sin degradacion de rendimiento superior al 20% en tiempos de respuesta de endpoints criticos.	<i>P95 latencia <= 3.6 s (120% de baseline) con 50 usuarios concurrentes simulados.</i>	Media
RNF-009	Mantenibilidad	El codigo de entrenamiento ML (ml/training/) no debe importar modulos del directorio backend/. La separacion de capas definida en CLAUDE.md debe respetarse.	<i>0 imports de "backend" en ml/training/. Verificado con: grep -r "from backend" ml/training/</i>	Alta
RNF-010	Mantenibilidad	Todas las migraciones de esquema de BD deben realizarse mediante Alembic. No se permite modificacion manual de la BD en produccion.	<i>100% de cambios de esquema en backend/migrations/ versionados con Alembic. Sin ALTER TABLE manual en produccion.</i>	Alta
RNF-011	Usabilidad	La interfaz de comandos de voz debe responder al agricultor en espanol con latencia total (audio -> respuesta TTS) menor a 8 segundos.	<i>Latencia total pipeline voz <= 8 s (Whisper + Ollama + accion + TTS). Medicion end-to-end.</i>	Media
RNF-012	Usabilidad	El sistema debe mostrar mensajes de error en espanol comprensibles para agricultores con bajo dominio tecnico. No se deben exponer mensajes de error internos del servidor.	<i>0 stack traces o errores HTTP crudos visibles en la UI. Todos los errores mapeados a mensajes en espanol.</i>	Alta
RNF-013	Precision Agro.	El motor FAO-56 Penman-Monteith debe producir resultados de ETo con error menor al 5% respecto a los valores de referencia de Allen et al. 1998 para datos climaticos del Valle del Yaqui.	<i>MAE(ETo_calculado, ETo_referencia) <= 5% en dataset de validacion. Verificado en TestPropagar (51 tests).</i>	Alta

4. Matriz de Trazabilidad de Requerimientos

La siguiente matriz relaciona cada caso de uso funcional con los módulos del sistema que lo implementan, los tests que lo validan y la deuda técnica asociada.



ID	Modulo Backend	Modulo Frontend	Tests	Estado	Deuda Tecnica
CU-MILP-001	POST /api/parcelas (db_api.py)	map_engine.js ui_tabs.js	TestRegistroParcela	Funcional	—
CU-MILP-002	GET /api/balance_hidrico (riego_api.py) core/balance_hidrico.py	ui_tabs.js (Tab Riego)	TestPropagar (9 tests) TestFAO56 unitarios	Funcional	—
CU-MILP-003	PATCH /recomendaciones/{id}/feedback db_api.py, riego_api.py	ui_tabs.js (Tab Riego)	TestFeedbackLoop (7 casos)	Funcional	—
CU-MILP-004	POST /api/voz/transcribir voice_endpoint.py	voice_client.js	—	Path traversal activo	Fase A #2
CU-MILP-005	GET /api/parcelas/geojson db_api.py + PostGIS	map_engine.js + Leaflet 1.9.4	TestGeoJSON	Funcional	—
CU-MILP-006	GET /api/parcelas/{id}/forecast core/eto_forecast.py	bi_dashboard.js (Tab Pronostico)	TestEtoForecast	Funcional	Fase B (train)
RNF-003/004	backend/.env voice_endpoint.py	—	—	Pendiente	Fase A #1, #2
RNF-009	ml/training/ (separacion capas)	—	—	Estructurado	Re-exports tmp